

AFRILANG Stdlib

std/c/gamepathfind

Bibliothèque AFRILANG `std/c/gamepathfind` (niveau complex, catégorie complex). Elle expose 6 fonction(s)
: bfsPath, pat

```
`import "std/c/gamepathfind" · `use gamepathfind`
```

- `bfsPath(grid list number, width number, start number, goal number) ? list number`
- `pathLen(grid list number, width number, start number, goal number) ? number`
- `lineOfSight(grid list number, width number, x0 number, y0 number, x1 number, y1 number) ? bool`
- `smoothPath(path list number, grid list number, width number) ? list number`
- `waypoints(path list number, step number) ? list number`
- `navCost(grid list number, width number, start number, goal number) ? number`

Category: complex | Tier: complex | Functions: 6

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/gamepathfind` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : bfsPath, pat

```
`import "std/c/gamepathfind"` . `use gamepathfind`  
- `bfsPath(grid list number, width number, start number, goal number) ? list number`  
- `pathLen(grid list number, width number, start number, goal number) ? number`  
- `lineOfSight(grid list number, width number, x0 number, y0 number, x1 number, y1 number) ? bool`  
- `smoothPath(path list number, grid list number, width number) ? list number`  
- `waypoints(path list number, step number) ? list number`  
- `navCost(grid list number, width number, start number, goal number) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/gamepathfind"  
use gamepathfind
```

Niveau : complex · Catégorie : Modules complex (c/)

Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? bfsPath(grid list number, width number, start number, goal number) ? list  
? pathLen(grid list number, width number, start number, goal number) ? number  
? lineOfSight(grid list number, width number, x0 number, y0 number, x1 number, y1 number) ? bool  
? smoothPath(path list number, grid list number, width number) ? list  
? waypoints(path list number, step number) ? list  
? navCost(grid list number, width number, start number, goal number) ? number
```

4. Exemples d'utilisation

```
import "std/c/gamepathfind"  
use gamepathfind  
  
create result = bfsPath(/* args */)   
say result  
  
create result = pathLen(/* args */)   
say result  
  
create result = lineOfSight(/* args */)   
say result  
  
create result = smoothPath(/* args */)   
say result  
  
create result = waypoints(/* args */)   
say result  
  
create result = navCost(/* args */)   
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/gamepathfind"` en tête de fichier.  
? Lancez `afrilang check` avant de livrer.  
? Combinez avec les modules stdlib de la même catégorie.  
? Voir /stdlib/ pour le source live et les démos playground.  
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module gamepathfind  
  
function bfsPath(grid list number, width number, start number, goal number) returns list number  
end
```

```
function pathLen(grid list number, width number, start number, goal number) returns number
end

function lineOfSight(grid list number, width number, x0 number, y0 number, x1 number, y1 number)
returns bool
end

function smoothPath(path list number, grid list number, width number) returns list number
end

function waypoints(path list number, step number) returns list number
end

function navCost(grid list number, width number, start number, goal number) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/gamepathfind` (tier complex, category complex). Exports 6 function(s): bfsPath, pathLen, lineOfS

```
`import "std/c/gamepathfind"` . `use gamepathfind`  
- `bfsPath(grid list number, width number, start number, goal number) ? list number`  
- `pathLen(grid list number, width number, start number, goal number) ? number`  
- `lineOfSight(grid list number, width number, x0 number, y0 number, x1 number, y1 number) ? bool`  
- `smoothPath(path list number, grid list number, width number) ? list number`  
- `waypoints(path list number, step number) ? list number`  
- `navCost(grid list number, width number, start number, goal number) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/gamepathfind"  
use gamepathfind
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? bfsPath(grid list number, width number, start number, goal number) ? list  
? pathLen(grid list number, width number, start number, goal number) ? number  
? lineOfSight(grid list number, width number, x0 number, y0 number, x1 number, y1 number) ? bool  
? smoothPath(path list number, grid list number, width number) ? list  
? waypoints(path list number, step number) ? list  
? navCost(grid list number, width number, start number, goal number) ? number
```

4. Usage examples

```
import "std/c/gamepathfind"  
use gamepathfind  
  
create result = bfsPath(/* args */)   
say result  
  
create result = pathLen(/* args */)   
say result  
  
create result = lineOfSight(/* args */)   
say result  
  
create result = smoothPath(/* args */)   
say result  
  
create result = waypoints(/* args */)   
say result  
  
create result = navCost(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/gamepathfind"` at the top of your file.  
? Run `afirang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module gamepathfind  
  
function bfsPath(grid list number, width number, start number, goal number) returns list number  
end
```

```
function pathLen(grid list number, width number, start number, goal number) returns number
end

function lineOfSight(grid list number, width number, x0 number, y0 number, x1 number, y1 number)
returns bool
end

function smoothPath(path list number, grid list number, width number) returns list number
end

function waypoints(path list number, step number) returns list number
end

function navCost(grid list number, width number, start number, goal number) returns number
end
end
```